



Introduction to Machine Learning (CS 436/536)

Lecture-2: Introduction to Machine Learning

Email: arakin@binghamton.edu

Grading Policy:

- Everyday Quiz (5%)
- Take-Home Assignments (20%)
- Paper Critique (10%)
- Final Project (20%)
- Class Test/Exam (30%)
- Final Exam (15%)

Weekly Lectures:

Week	Date	Lecture?	Topics
1	8/18/2025 (Tuesday)	Yes	Introduction (Lecture-1)
	8/20/2025 (Thursday)	Yes	Machine Learning Basics (Lecture-2)
2	8/25/2025 (Tuesday)	Yes	Mathematical Foundation (Lecture-3)
	8/27/2025 (Thursday)	Yes	Mathematical Foundation (Lecture-3) Code Basics (Python and PyTorch) (Lecture-4)
3	9/1/2025 (Tuesday)	Yes	Quiz-1
	9/4/2025 (Thursday)	Yes	Linear Regression (Lecture-5)

Weekly Lectures:

Week	Date	Lecture?	Topics
4	9/8/2025 (Tuesday)	No	Monday Class meet
	9/10/2025 (Thursday)	Yes	Linear Regression (Lecture-6)
5	9/15/2025 (Tuesday)	Yes	Logistic Regression (Lecture-7)
	9/17/2025 (Thursday)	Yes	Evaluation (Lecture-8)
6	9/23/2025 (Tuesday)	yes	Quiz-2
	9/25/2025 (Thursday)	Yes	Neural Network and Backpropagation (Lecture 9 & 10)

Weekly Lectures :

Week	Date	Lecture?	Topics
10	10/20/2025 (Tuesday)	Yes	Probability Based Learning (Lecture-14)
	10/22/2025 (Thursday)	Yes	Decision Trees and Clustering (Lecture 15 & 16)
11	10/27/2025 (Tuesday)	Yes	GAN and Self-supervised learning (Lecture 15 & 16) Dimensionality Reduction (Lecture-17)
	10/29/2025 (Thursday)	Yes	Quiz-4
12	11/3/2025 (Tuesday)	Yes	Dimensionality Reduction (Lecture-17) Hidden Markov Model (Lecture-18)
	11/5/2025 (Thursday)	Yes	Advanced topics: LLMs and AI Security (Lecture-19)

Weekly Lectures :

Week	Date	Lecture?	Topics
13	11/10/2025 (Tuesday)	Yes	Final Project Demo
	11/12/2025 (Thursday)	Yes	Final Project Demo
14	11/17/2025 (Tuesday)	Yes	Final Project Demo
	11/19/2025 (Thursday)	Yes	Quiz-5
15	11/24/2025 (Tuesday)	No	Friday's classes meet
	11/26/2025 (Thursday)	No	Thanksgiving break
16	12/1/2025 (Tuesday)	Yes	Final Project Demo
	12/3/2025 (Tuesday)	Yes	Final Project Demo

TA Details:

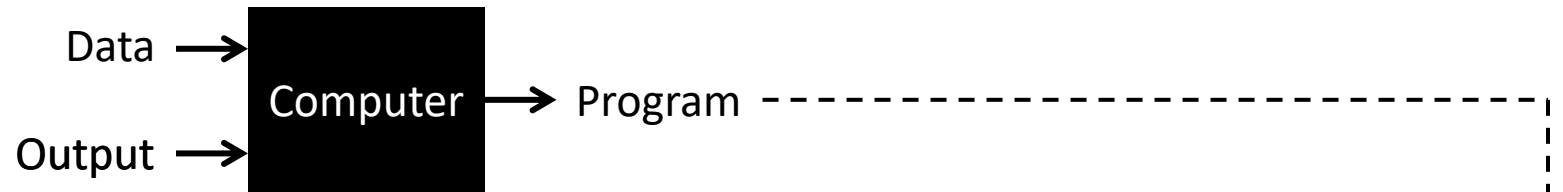
Mohaiminul Al Nahian

- Office hours: Wednesday 1-3 pm
- Office: N00 (TA offices)
- Email: malnahian@binghamton.edu

What is Machine Learning?

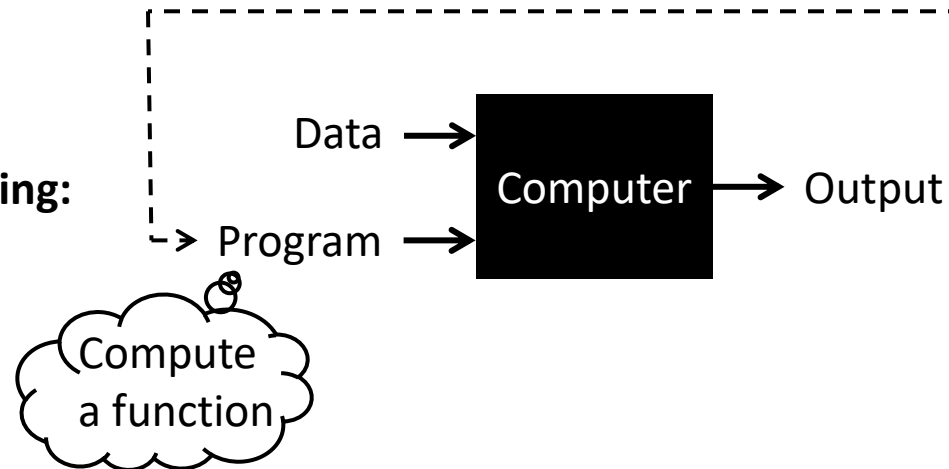
“computer programs that automatically improve their performance through experience” – Herbert Simon

Machine Learning:



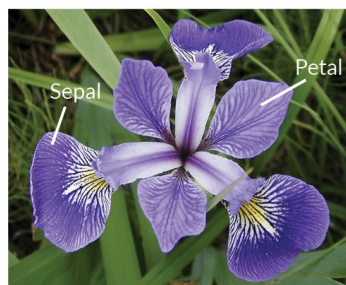
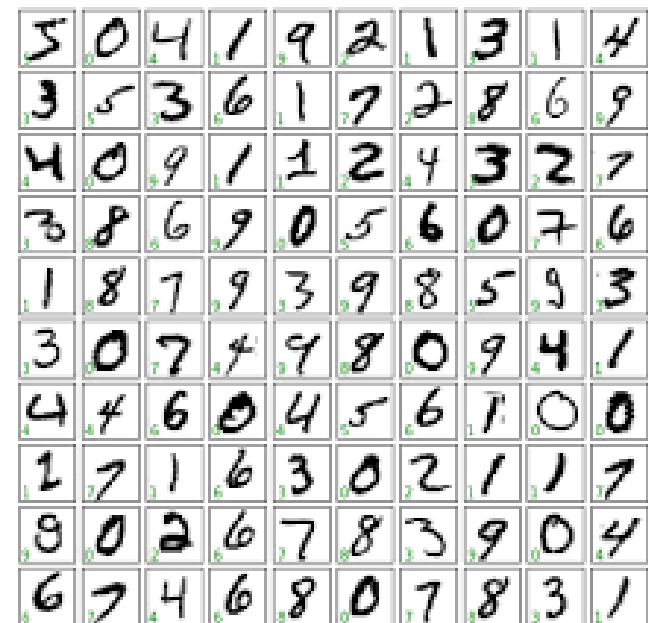
“... the ability to learn without being explicitly programmed” – Arthur Samuel

Traditional Programming:



What it is not:
The design approach

Why? Huh!? What is it good for?



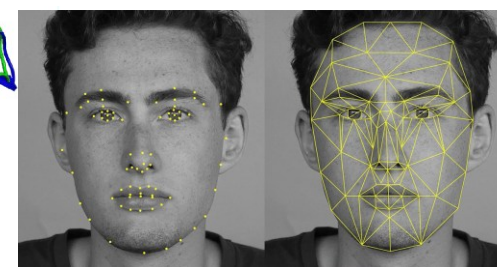
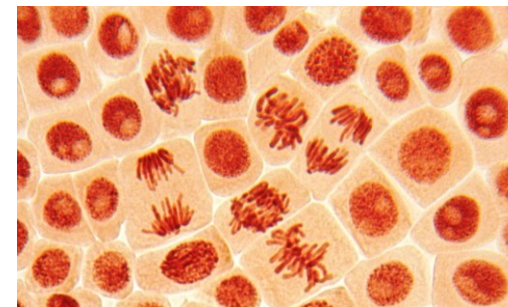
Iris Versicolor



Iris Setosa



Iris Virginica



- how to
- how to **care for a ficus tree**
- how to **propagate ficus elastica**
- how to **propagate ficus benjamina**
- how to **get job in machine learning**

Learning to Solve Hard Problems with Data

Some Other Applications

- Recognizing patterns
 - Facial identities or facial expressions
 - Handwritten or spoken words
 - Medical images
- Generating patterns
 - Generating images or motion sequences
- Recognizing anomalies
 - Unusual credit card transactions
 - Unusual patterns of sensor readings in a nuclear power plant

Objective

- Write a program that predicts tomorrow's weather given geographic information, satellite images, and a trailing window of past weather.
- Write a program that takes in a question, expressed in free-form text, and answers it correctly.
- Write a program that given an image can identify all the people it contains, drawing outlines around each.
- Write a program that presents users with products that they are likely to enjoy but unlikely, in the natural course of browsing, to encounter.

Credit Approval

- Using salary, debt, years in residence, etc., approve for credit or not
- Nobody has an optimal credit approval formula
- But banks have data
 - Customer information
 - Credit history

age	33 years
salary	50,000
debt	27,500
years employed	1
years at residence	2
...	...

Approve for credit?

Outline

1. What is Learning?

“Learning is a process by which a system improves performance from experience”

- Herbert Simon

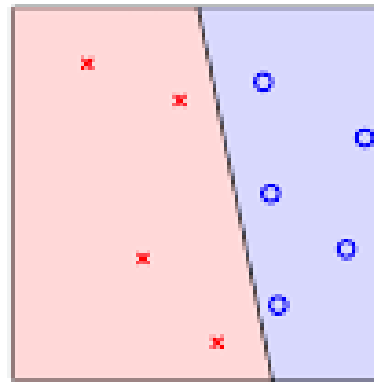
2. Can we Learn?

3. How to Learn?

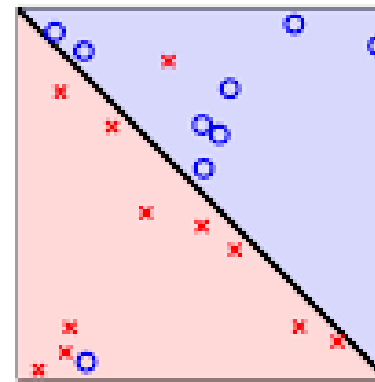
4. How to Learn well?

5. Advanced Techniques.

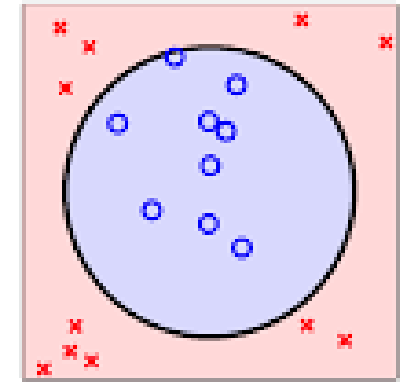
6. Other Learning Paradigms.



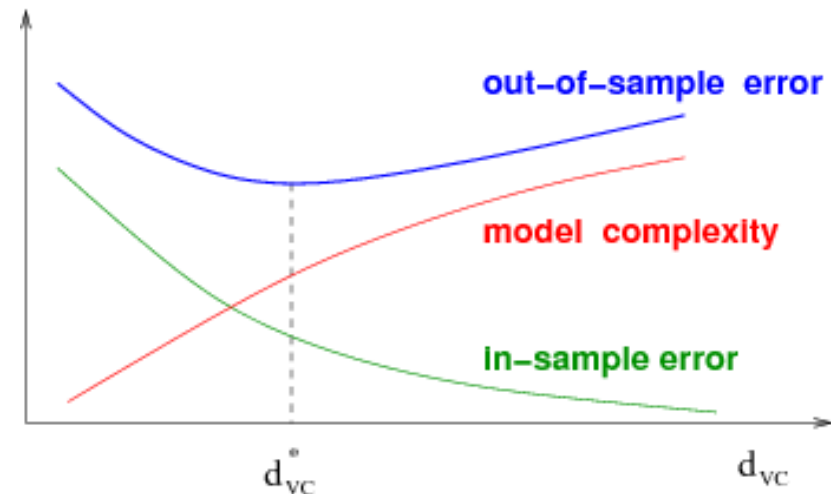
(linear separable)



(not linear separable)



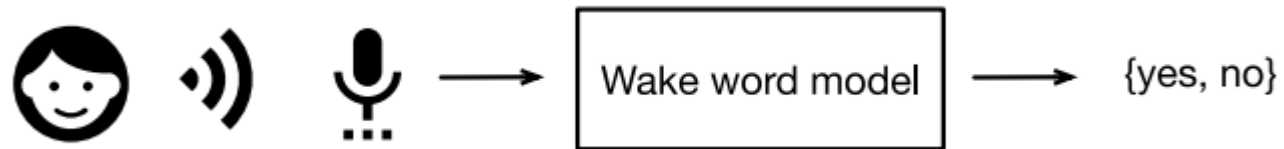
(not linear separable)



How to Learn?

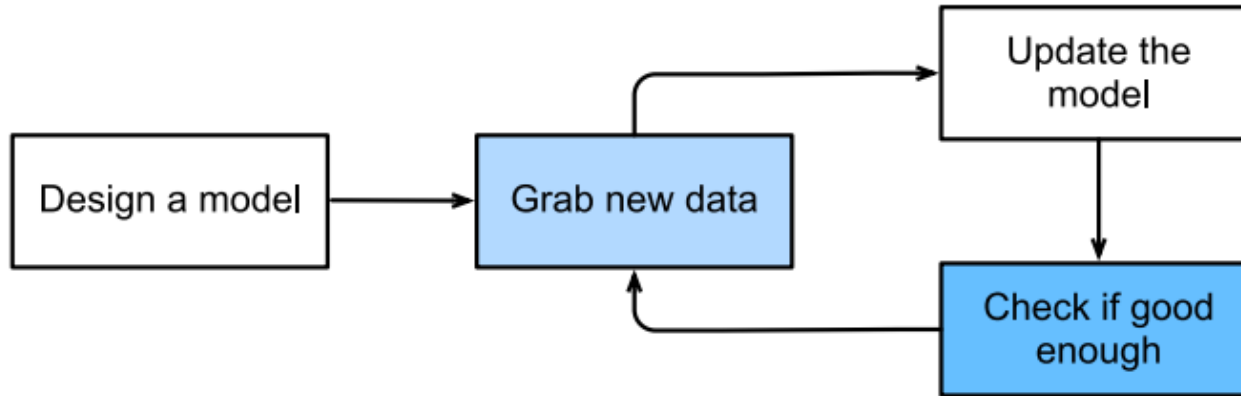
- Memorization
 - Accumulate all the facts in a large lookup table
- Generalization
 - Deduce new facts from old facts
 - Recognize a pattern and use it to *make a prediction*
- A **pattern** exists
- The **pattern** is **UNKNOWN**
- But we have data

A Motivating Example:

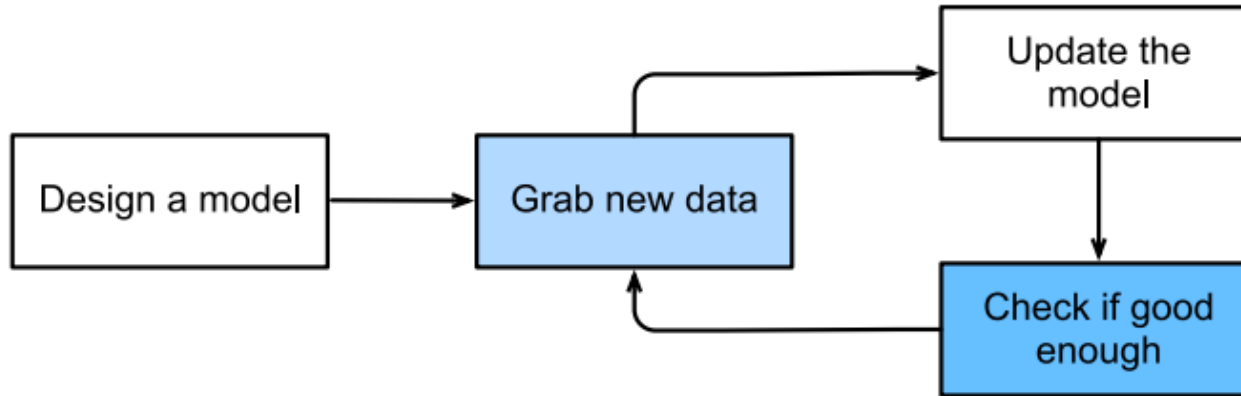


- ☐ Hey Siri !
- ☐ Okay Google!
- ☐ Hey Alexa!
- ☐ Hey Mercedes!

Learning Process:



Learning Process:



- Start off with a randomly initialized model that cannot do anything useful.
- Grab some of your data (e.g., audio snippets and corresponding {yes, no} labels).
- Tweak the knobs so the model performs better with respect to those examples.
- Repeat Step 2 and 3 until the model is awesome.

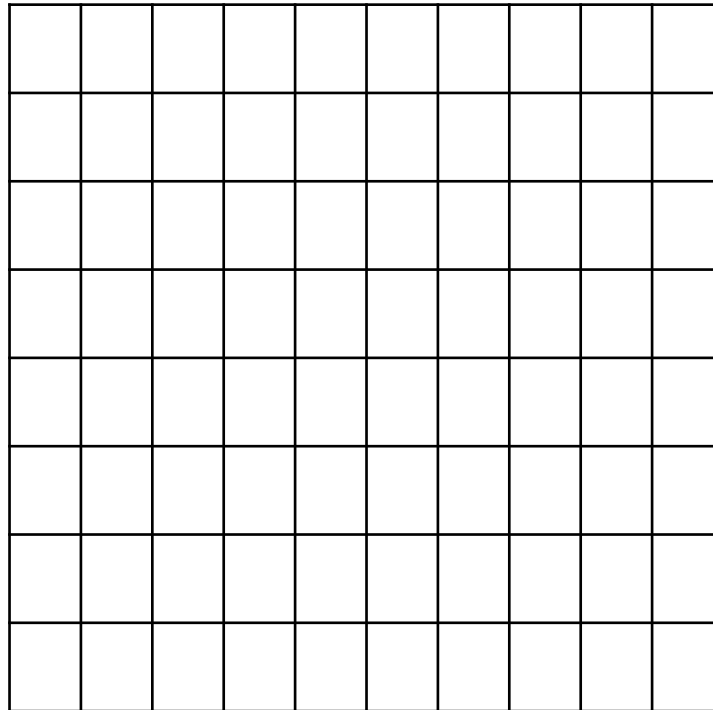
Key Learning Components:

- A data that we can learn from
- A model that can transform the data
- An objective function that quantifies how well (or badly) the model is doing.
- An algorithm to adjust the model's parameters to optimize the objective function.

What is Data?



What is Data?



What is Data?

[illegible]

What is Data?



0.9	0.5								

In the supervised learning problems, we want to predict special attribute that is designated as the label (or target).

Some General Ideas of Data

- The more data we have, the easier our job becomes
- Not all data can easily be represented as fixed-length vectors
- It is not enough to have lots of data
- To process it cleverly, we need the right data

Model

- Most machine learning involves transforming the data in some sense.
- Computational machinery for ingesting data of one type, and spitting out predictions of a possibly different type
- We are interested in statistical models that can be estimated from data.

Model

- Most machine learning involves transforming the data in some sense.
- Computational machinery for ingesting data of one type, and spitting out predictions of a possibly different type
- We are interested in statistical models that can be estimated from data

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Model

- Most machine learning involves transforming the data in some sense.
- Computational machinery for ingesting data of one type, and spitting out predictions of a possibly different type
- We are interested in statistical models that can be estimated from data.

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Mean = 5.5

Model

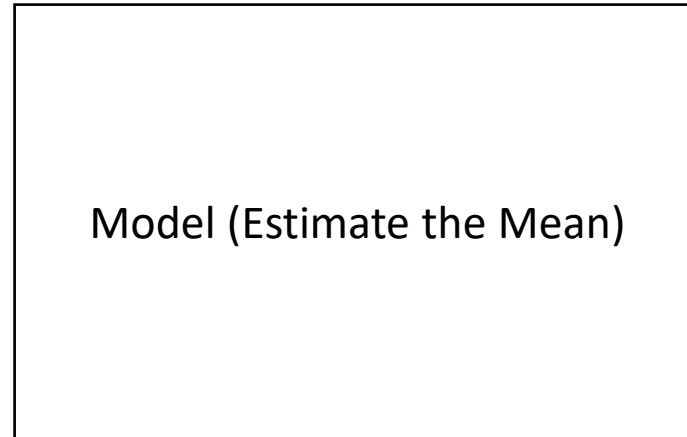
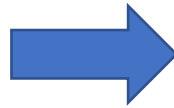
- Most machine learning involves transforming the data in some sense.
- Computational machinery for ingesting data of one type, and spitting out predictions of a possibly different type
- We are interested in statistical models that can be estimated from data.

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

$$\text{Mean} = 5.5 = (\sum \text{All Data Points}) / (\text{Total Data Point})$$

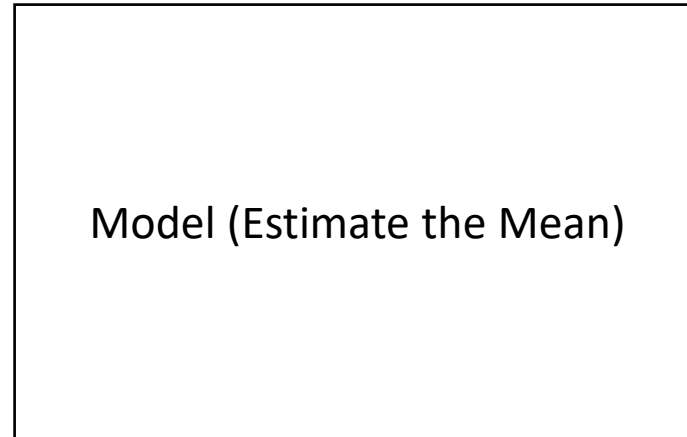
Combine The Data and Model

1
2
3
4
5
6
7
8
9



Combine The Data and Model

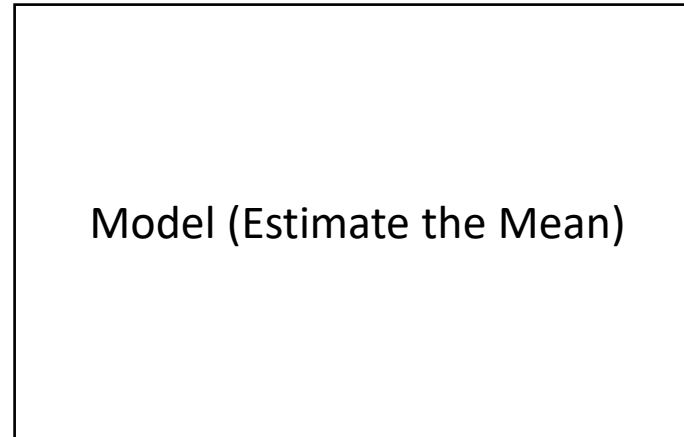
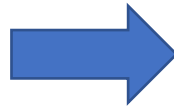
1
2
3
4
5
6
7
8
9



Output = 5.5

Combine The Data and Model

1
2
3
4
5
6
7
8
9

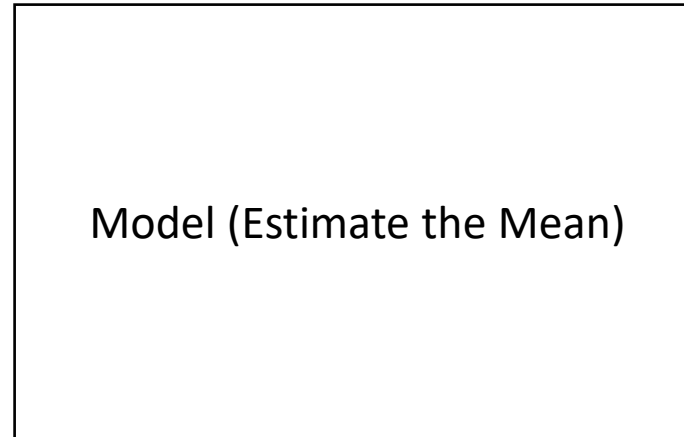
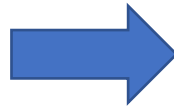


Output = 5.5

Wrong

Combine The Data and Model

1
2
3
4
5
6
7
8
9



Output = 5.5

How to Fix this?

Wrong

Objective Functions

- Formal measures of how good (or bad) our models are
- In our example it missed by 0.5
- By convention, we usually define objective functions so that lower is better
- Because lower is better, these functions are sometimes called loss functions
- Examples: Squared Error, Absolute Error and Cross-Entropy
- We learn the best values of our model's parameters by minimizing the loss incurred on a set consisting of some number of examples collected for training.

Optimization Algorithms

- An algorithm which can minimize our objective function
- Popular optimization algorithms for deep learning are based on an approach called gradient descent.
- Updates the parameter in the direction that may reduce the loss.

Credit Approval

- Using salary, debt, years in residence, etc., approve for credit or not
- Nobody has an optimal credit approval formula
- But banks have data
 - Customer information
 - Credit history

age	33 years
salary	50,000
debt	27,500
years employed	1
years at residence	2
...	...

Approve for credit?

Components of the Learning Problem

- age, salary, debt, years employed, ...
- Approve credit or not
- **True** relationship between x and y
- Data on customers

input $x \in \mathbb{R}^d = \mathcal{X}$

output $y \in \{-1, +1\} = \mathcal{Y}$

target function $f: \mathcal{X} \rightarrow \mathcal{Y}$
[unknown]

Optimal credit
approval function

data set $\mathcal{D}_N = (x_1, y_1), \dots (x_N, y_N)$
[$y_n = f(x_n)$]

$\mathcal{X}, \mathcal{Y}$ and $\mathcal{D}$ are *given* by the learning problem

The target function f is fixed but unknown

Learn the target function f from the data $\mathcal{D}$

What is Learning?

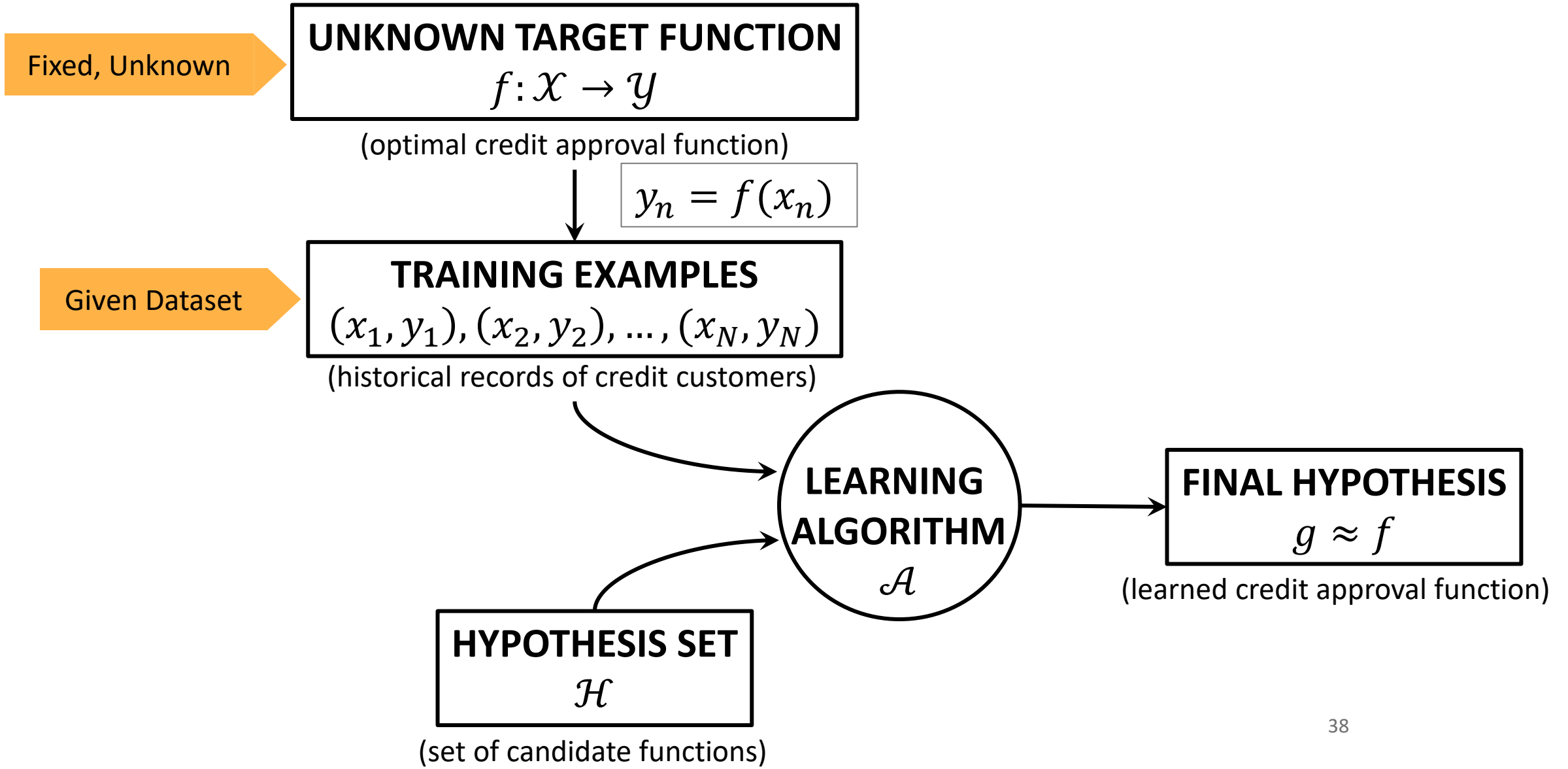
- Obtaining f
- Result of learning is an approximation of f

$$g: \mathcal{X} \rightarrow \mathcal{Y}$$

- Want:

$g \approx f$ i.e. $g(\mathbf{x}_*) \approx f(\mathbf{x}_*)$ where $\mathbf{x}_*$ is the next *test data point*

Learning Problem Setup



Learning

- Start with a set of candidate hypotheses $\mathcal{H}$ which likely represent f
 $\mathcal{H} = \{h_1, h_2, \dots\}$ The hypothesis set or *model*

- Select a hypothesis g from $\mathcal{H}$

Using a *learning algorithm*

- Use g for new customers

Hope that $g \approx f$

$\mathcal{X}, \mathcal{Y}$ and $\mathcal{D}$ are **given** by the learning problem

The target function f is **fixed but unknown**

We choose $\mathcal{H}$ and the learning algorithm

Learning

- Start with a set of candidate hypotheses $\mathcal{H}$ which likely represent f
 $\mathcal{H} = \{h_1, h_2, \dots\}$ The hypothesis set or *model*

- Select a hypothesis g from $\mathcal{H}$

A Decision Problem: What is the Criterion?

Using a *learning algorithm*

A Computational Problem

- Use g for new customers

Hope that $g \approx f$

$\mathcal{X}, \mathcal{Y}$ and $\mathcal{D}$ are **given** by the learning problem

The target function f is **fixed but unknown**

We choose $\mathcal{H}$ and the learning algorithm

A Simple Learning Model

- Input vector $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$
- Compute a “credit score” by giving importance weights to the different inputs: $creditscore = \sum_{i=1}^d w_i x_i$
- Decision rule:
 - If $creditscore > threshold$: **Approve** credit (good credit score)
 - If $creditscore < threshold$: **Deny** credit (poor credit score)
- How to choose the importance weights w_i ?
 - input x_i is important in deciding credit approval $\Rightarrow$ large w_i
 - input x_i has a beneficial effect to credit $\Rightarrow w_i > 0$ (weighs positively)
 - input x_i has an adverse effect on credit $\Rightarrow w_i < 0$ (weighs negatively)

A Simple Learning Model

- Decision rule:

- If $creditscore > threshold$: **Approve** credit (good credit score) $\Rightarrow$ output **+1**
- If $creditscore < threshold$: **Deny** credit (poor credit score) $\Rightarrow$ output **-1**

- Can be written formally as:

$$h(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) - threshold \right)$$

Simplifying a little...

$$h(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) + w_0 * 1 \right)$$

w_0 is a “bias weight” which corresponds to the threshold: Approve if $\sum_{i=1}^d w_i x_i > w_0$

A Simple Learning Model

$$h(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) + w_0 * 1 \right)$$

$$= \text{sign}(w_0 * 1 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d)$$

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix} \in \mathbb{R}^{d+1} \quad \mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \in 1 \times \mathbb{R}^d \text{ (where } x_0 = 1)$$

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$$

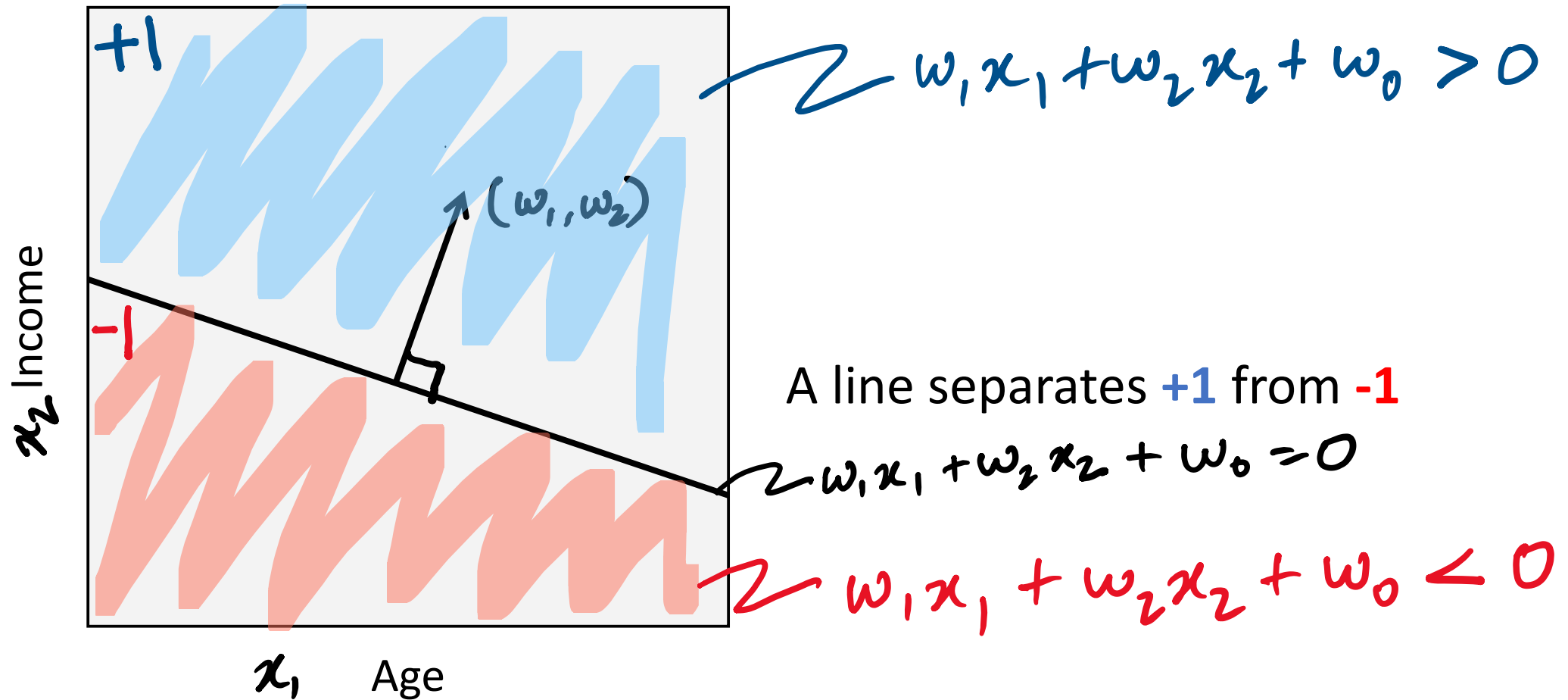
The Perceptron Hypothesis Set

- We define a hypothesis set $\mathcal{H}$

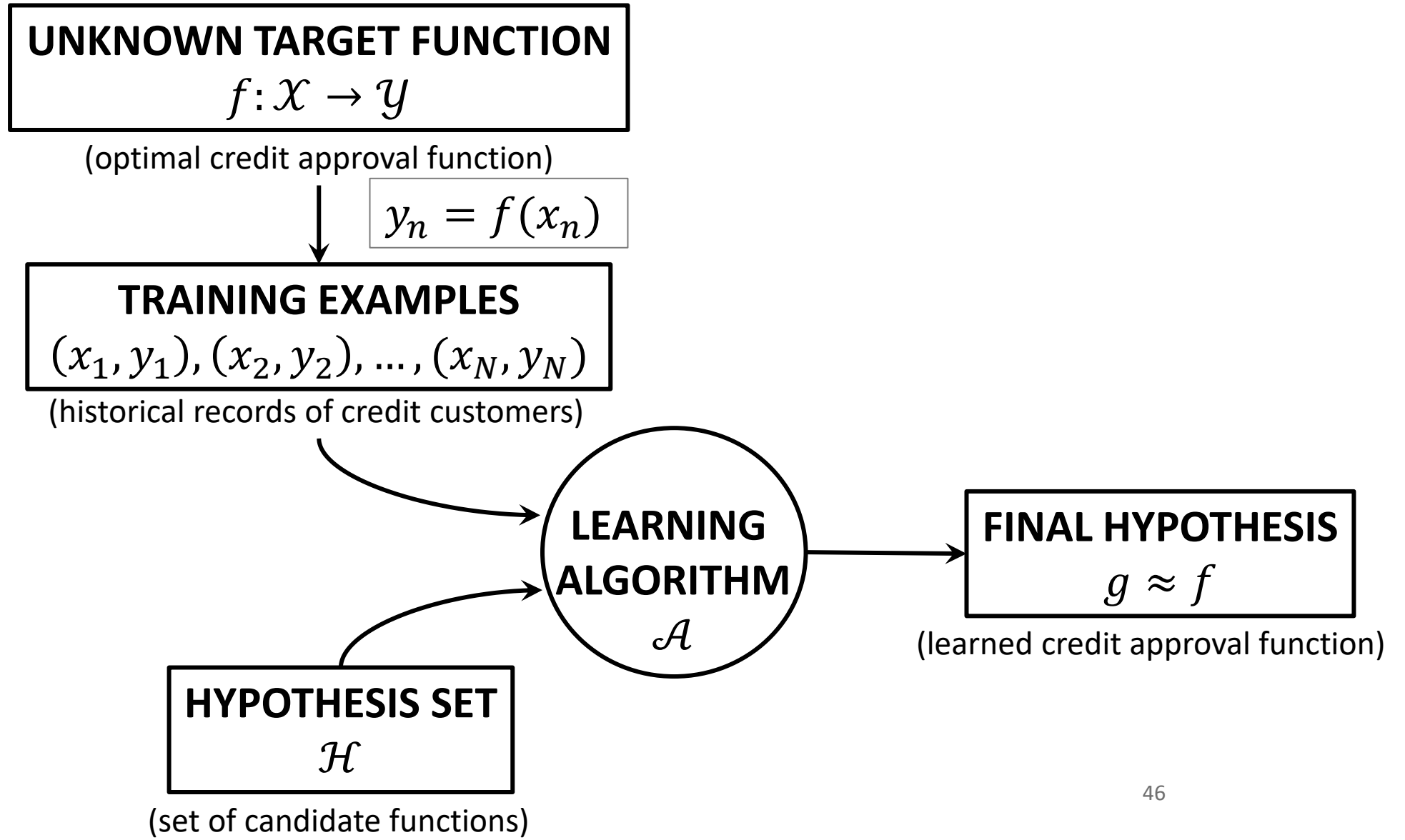
$$\mathcal{H} = \{h(x) = \text{sign}(\mathbf{w}^T \mathbf{x})\}$$

The ***perceptron*** or ***linear separator***

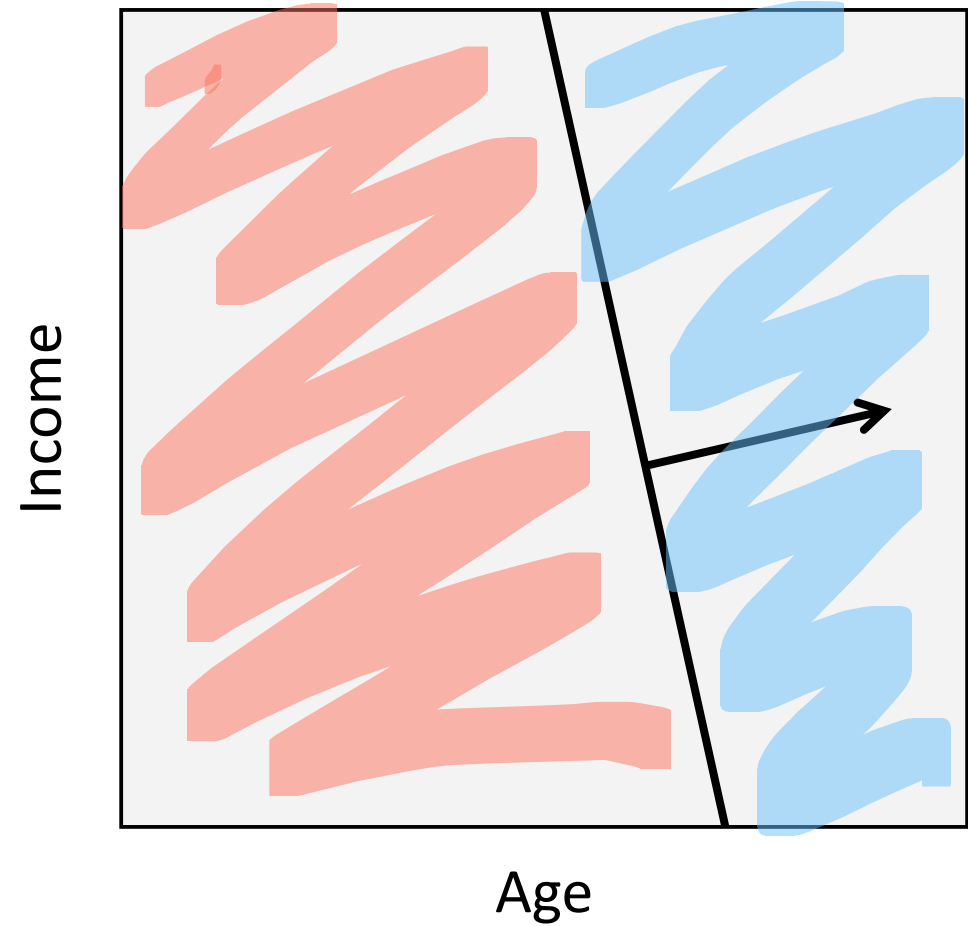
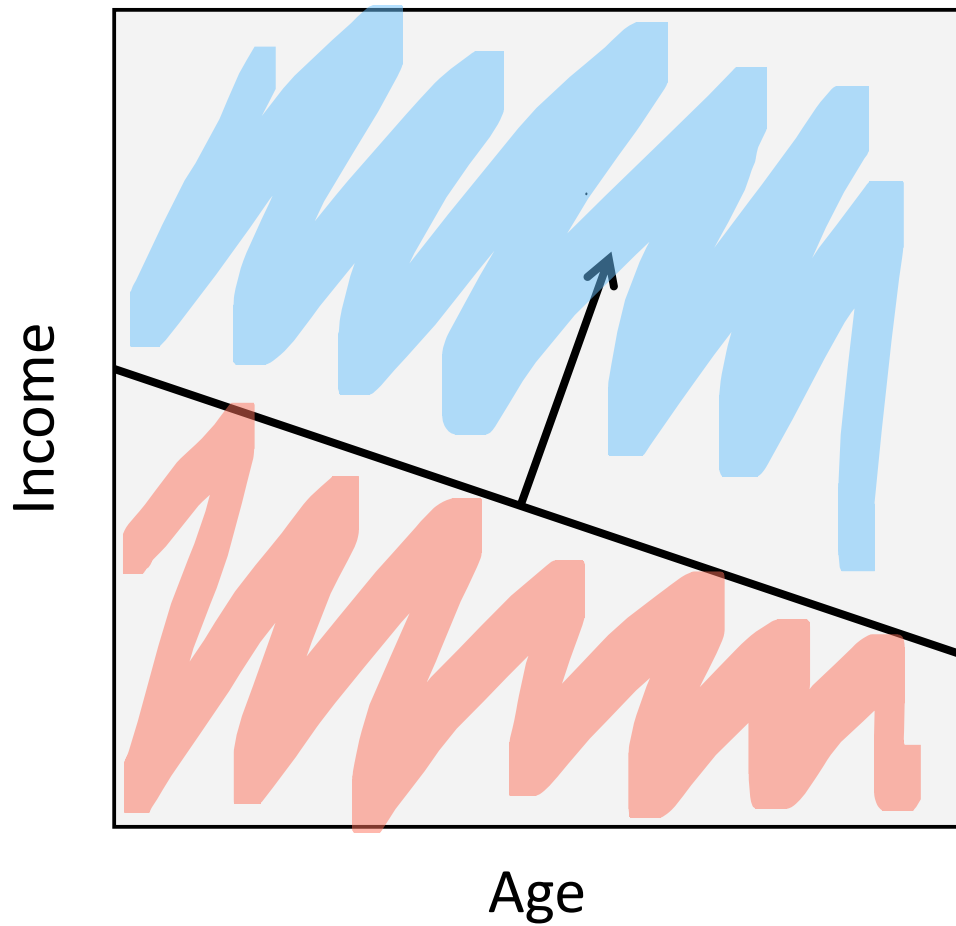
Geometry of The Perceptron in $\mathbb{R}^2$



Learning Problem Setup

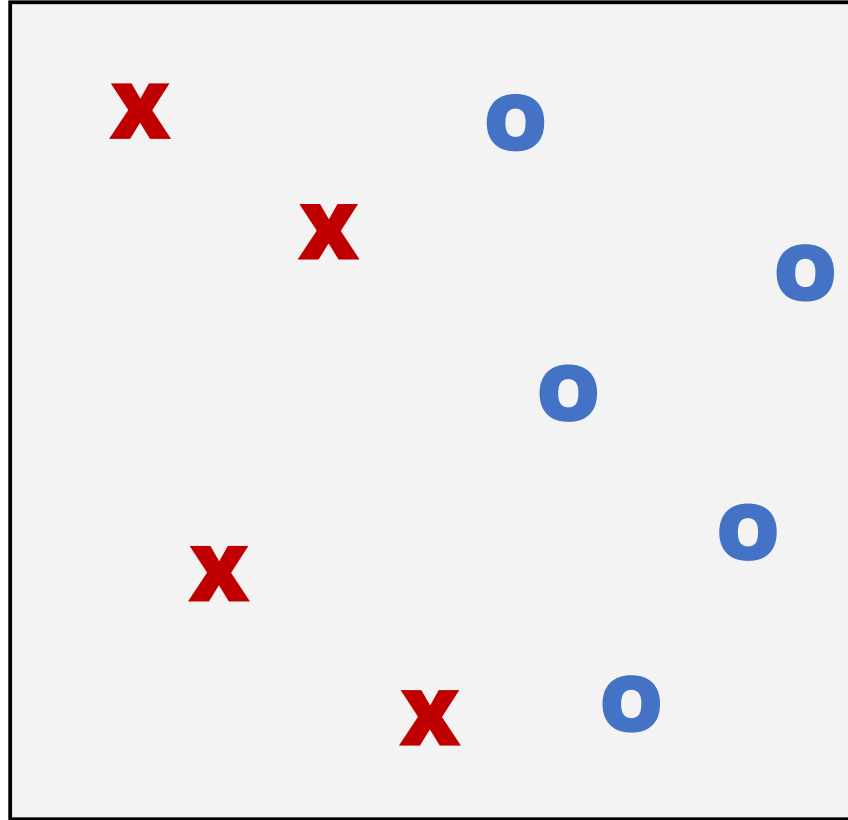


Geometry of The Perceptron in $\mathbb{R}^2$

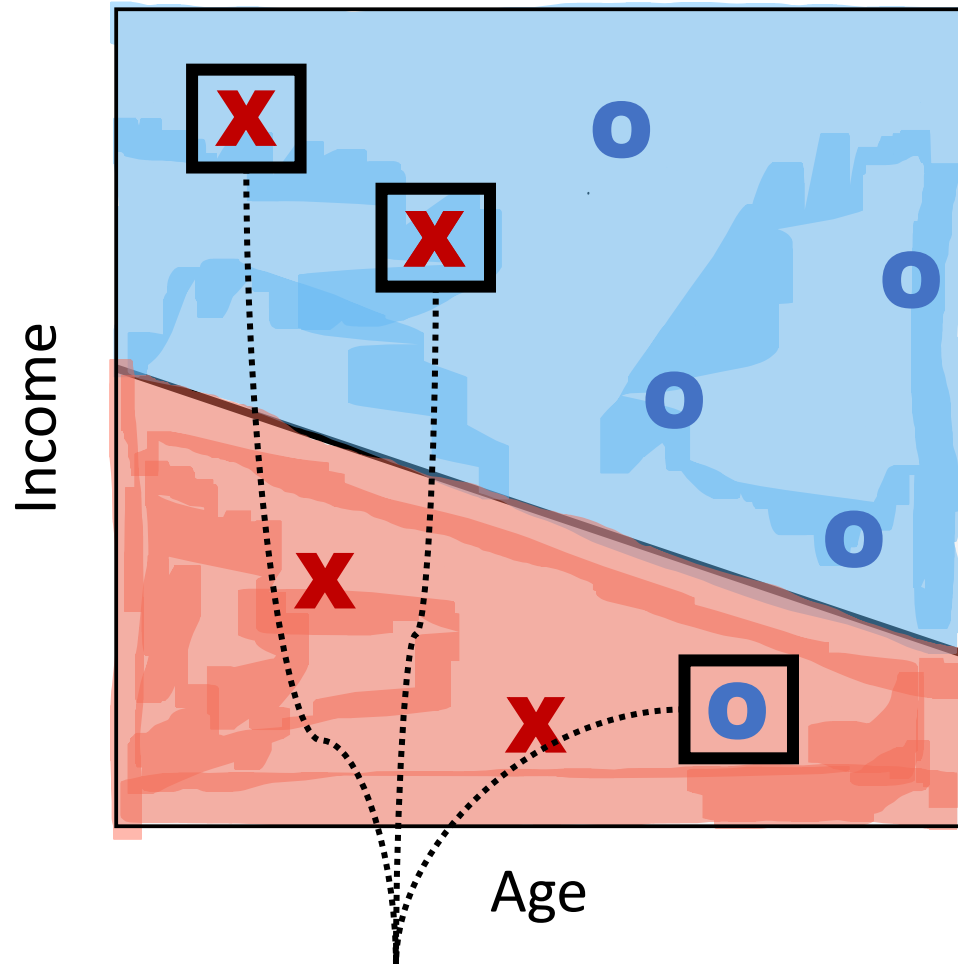


Which one to pick?

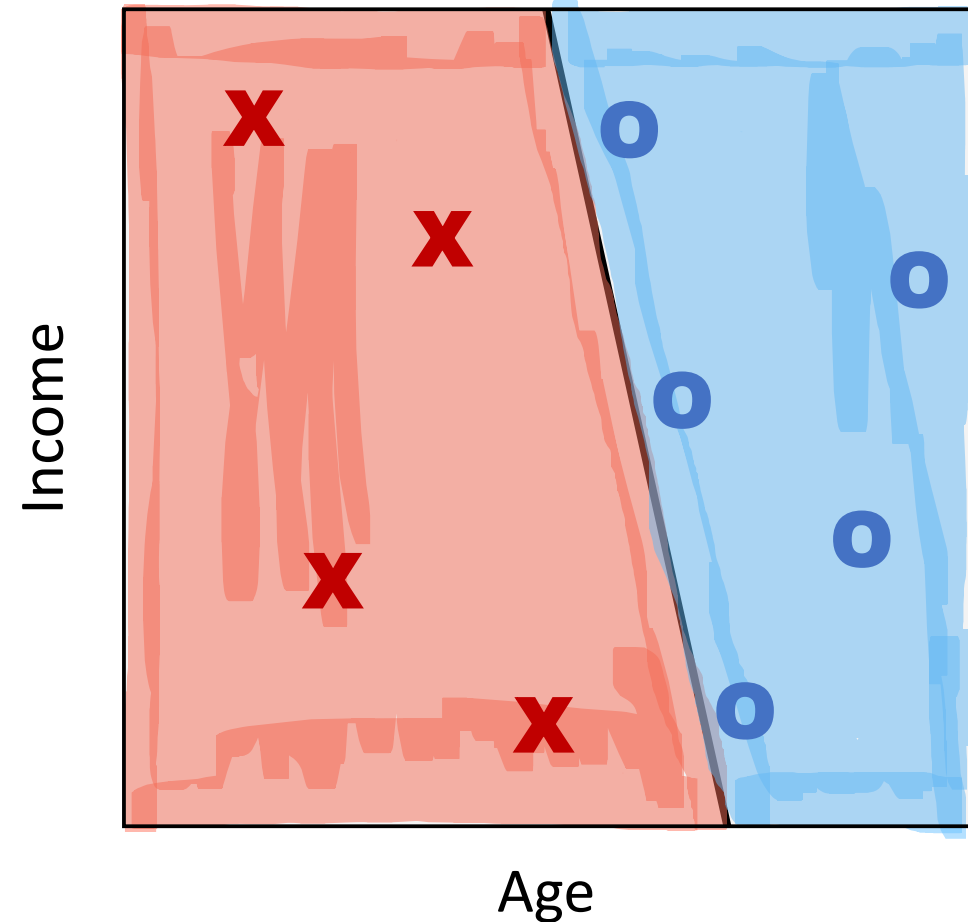
Using Data



Select a Hypothesis Using Data



misclassified data



perfectly classified

How to Learn a Final Hypothesis g from $\mathcal{H}$?

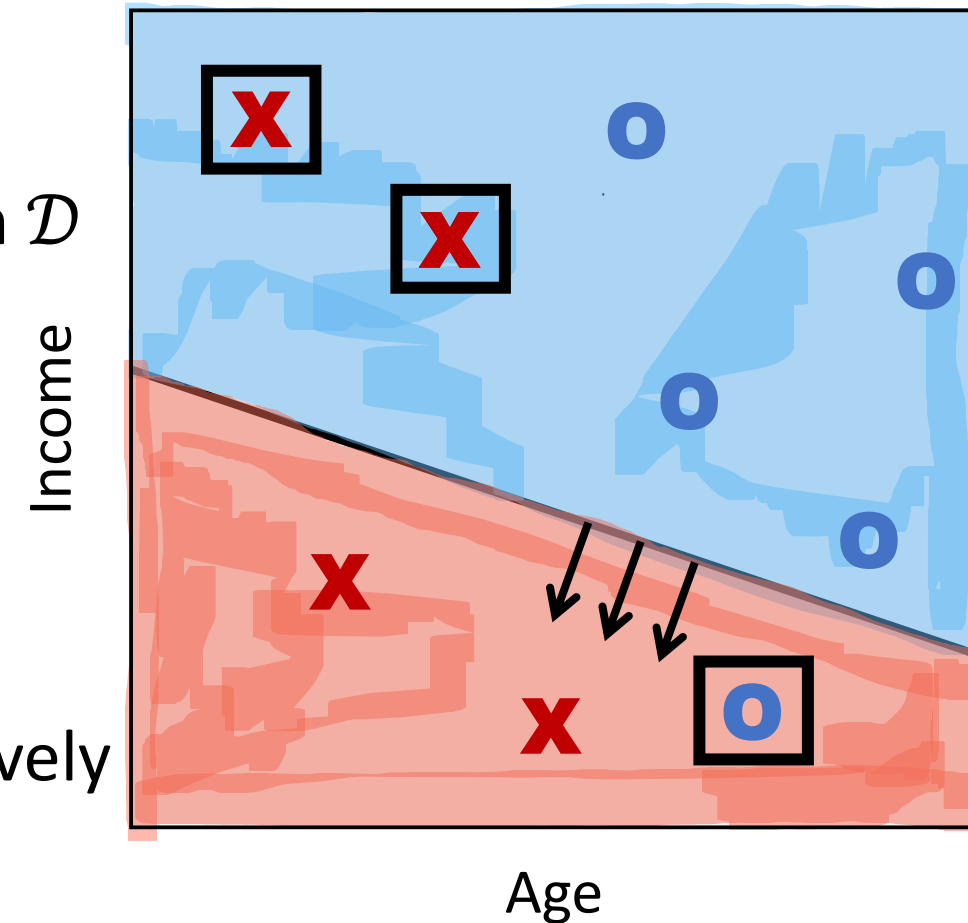
- Want: Select g from $\mathcal{H}$ so that $g \approx f$
- Certainly want $g \approx f$ on the dataset $\mathcal{D}$, i.e.,
$$g(\mathbf{x}_n) = y_n \text{ for each } (\mathbf{x}_n, y_n) \text{ in } \mathcal{D}$$

- But $\mathcal{H}$ is uncountably infinite

How to find g in the infinite hypothesis set $\mathcal{H}$?



Start with *some* weights and improve it iteratively



A Simple Learning Model

- Decision rule:

- If $creditscore > threshold$: **Approve** credit (good credit score) $\Rightarrow$ output **+1**
- If $creditscore < threshold$: **Deny** credit (poor credit score) $\Rightarrow$ output **-1**

- Can be written formally as:

$$h(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) - threshold \right)$$

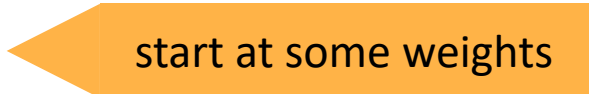
Simplifying a little...

$$h(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) + w_0 * 1 \right)$$

w_0 is a “bias weight” which corresponds to the threshold: Approve if $\sum_{i=1}^d w_i x_i > w_0$

The Perceptron Learning Algorithm (PLA)

A simple iterative algorithm

1. $\mathbf{w}(0) = \mathbf{0}$  start at some weights
2. **for** iteration $t = 1, 2, 3, \dots$ **do**
3. the weight vector is $\mathbf{w}(t)$
4. **from** $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)$ pick *any* misclassified example
5. let $(\mathbf{x}_*, y_*)$ be the misclassified example
 $\text{sign}(\mathbf{w}(t) \cdot \mathbf{x}_*) \neq y_*$
6. update the weights
 $\mathbf{w}(t + 1) = \mathbf{w}(t) + \text{eta} * y_* \mathbf{x}_*$  improve it
7. $t \leftarrow t + 1$

“incremental learning” one example at a time

The Perceptron Update Rule

- $\mathbf{w}(t + 1) = \mathbf{w}(t) + y_* \mathbf{x}_*$

Mistake when $y_* = +1$: Increase the score of $\mathbf{x}_*$

$$\begin{aligned}\mathbf{w}^T(t + 1)\mathbf{x}_* &= (\mathbf{w}^T(t) + \mathbf{x}_*^T)\mathbf{x}_* = \mathbf{w}^T(t)\mathbf{x}_* + \mathbf{x}_*^T\mathbf{x}_* \\ &> \mathbf{w}^T(t)\mathbf{x}_*\end{aligned}$$

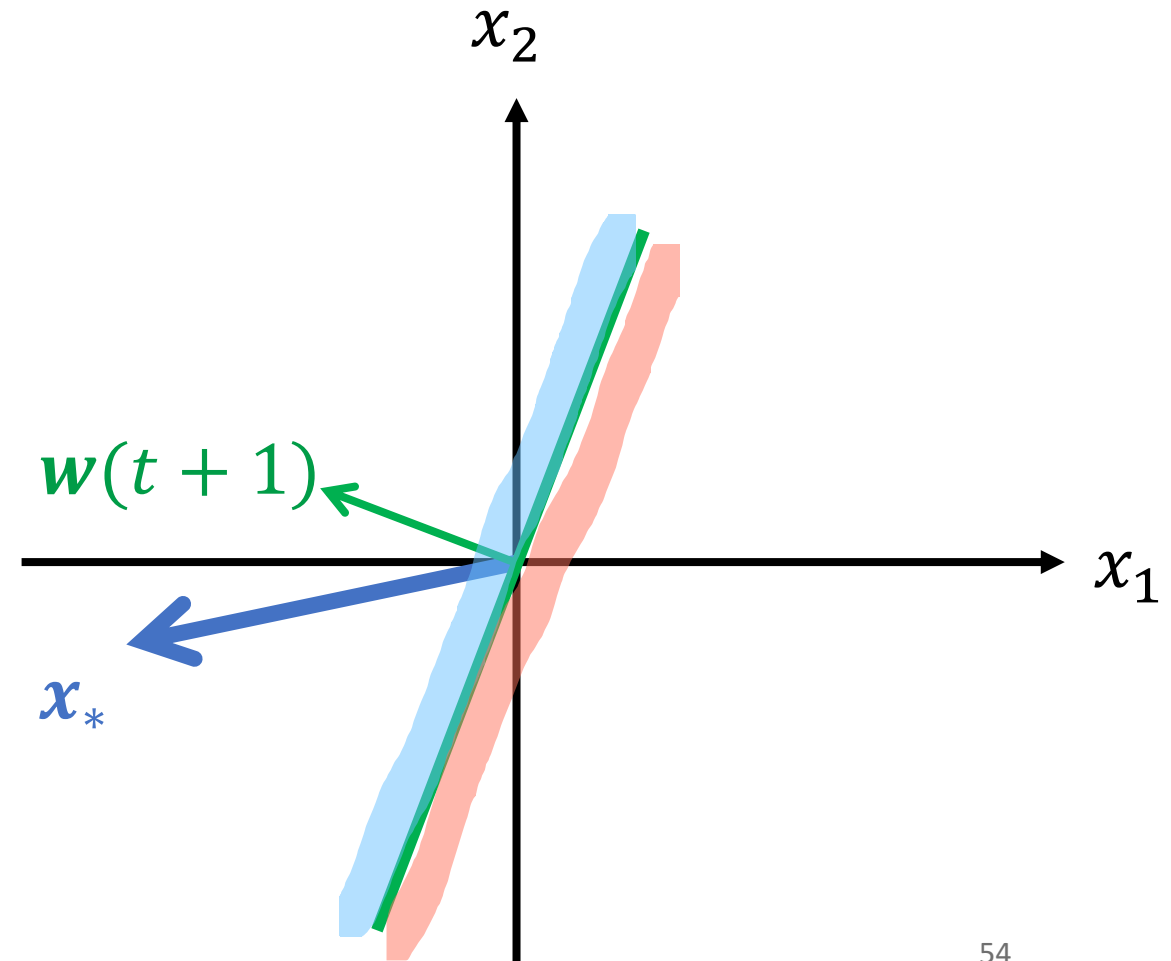
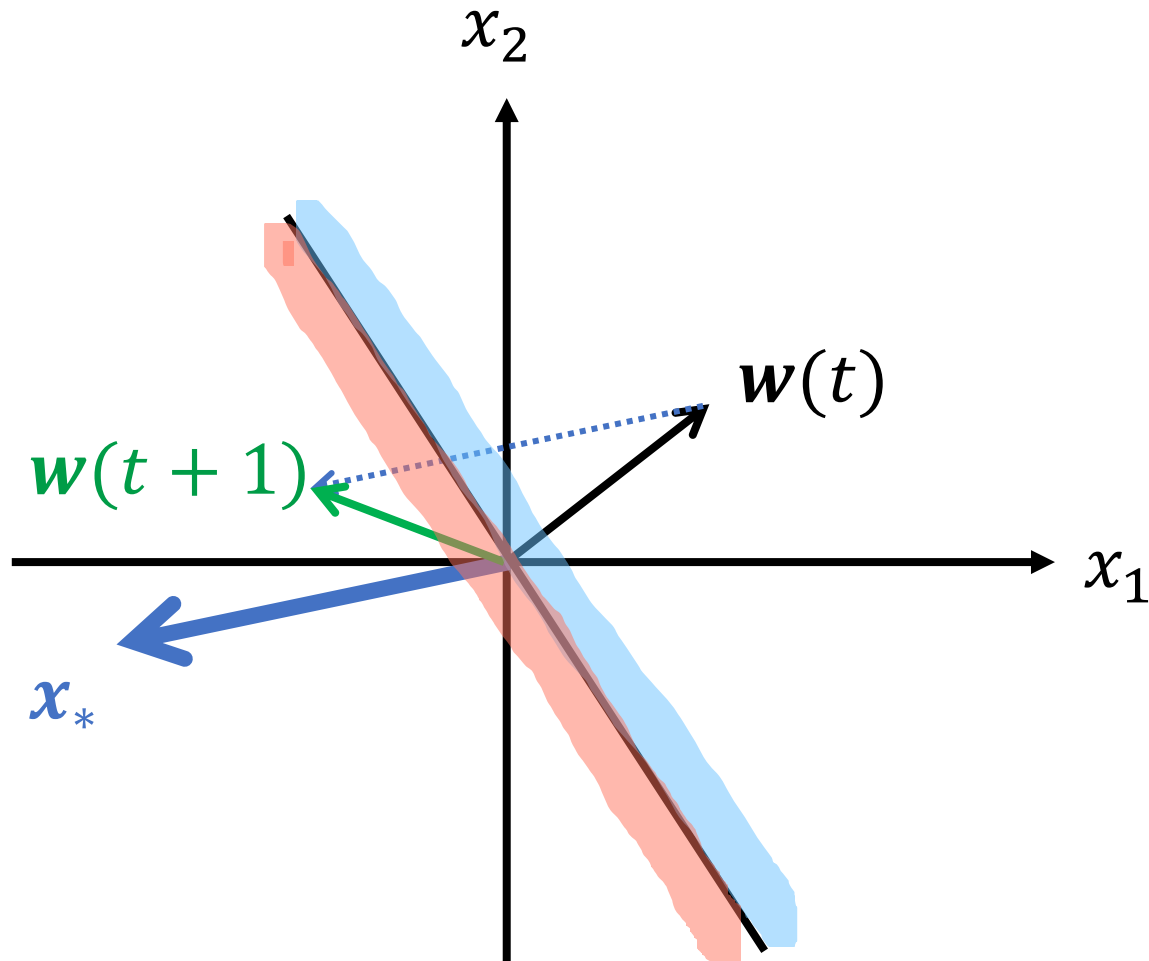
Mistake when $y_* = -1$: Decrease the score of $\mathbf{x}_*$

$$\begin{aligned}\mathbf{w}^T(t + 1)\mathbf{x}_* &= (\mathbf{w}^T(t) - \mathbf{x}_*^T)\mathbf{x}_* = \mathbf{w}^T(t)\mathbf{x}_* - \mathbf{x}_*^T\mathbf{x}_* \\ &< \mathbf{w}^T(t)\mathbf{x}_*\end{aligned}$$

Proof of convergence: [perc.converge.pdf \(columbia.edu\)](http://perc.converge.pdf(columbia.edu))

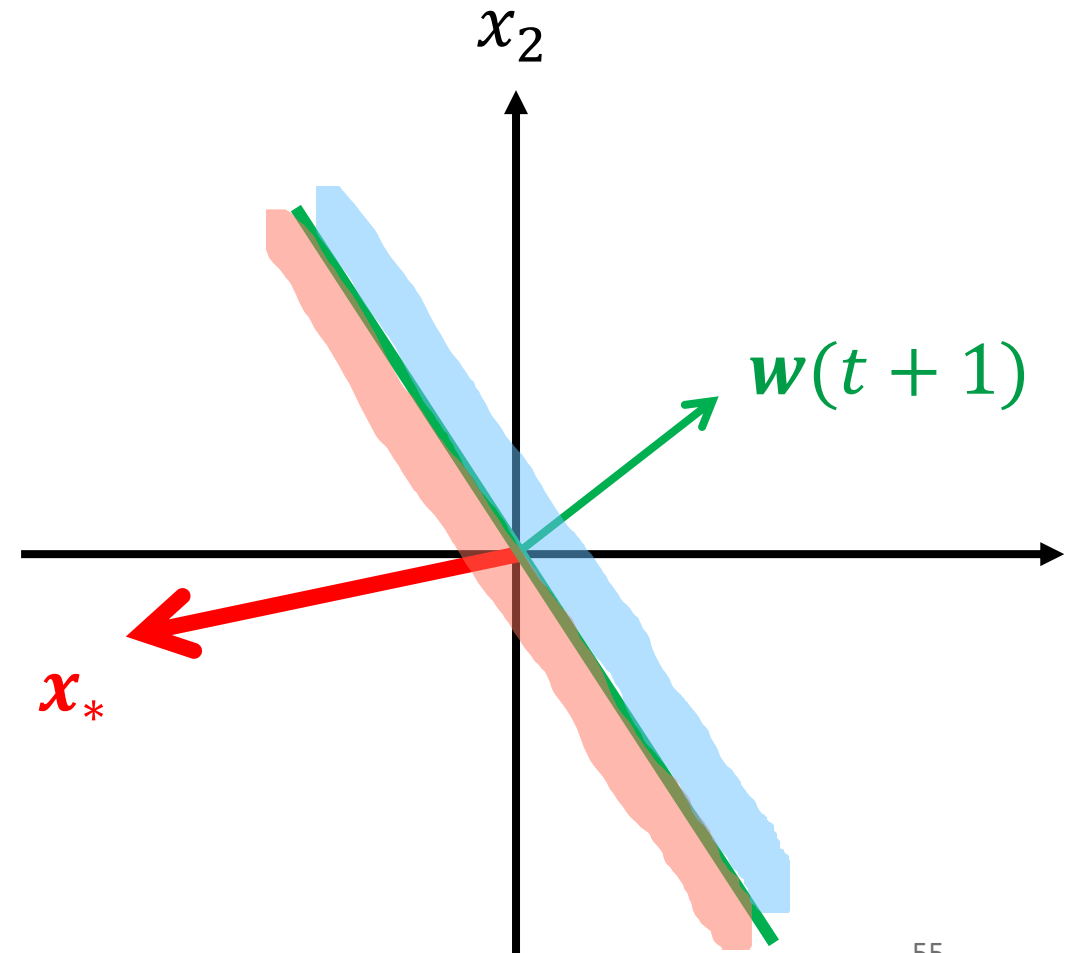
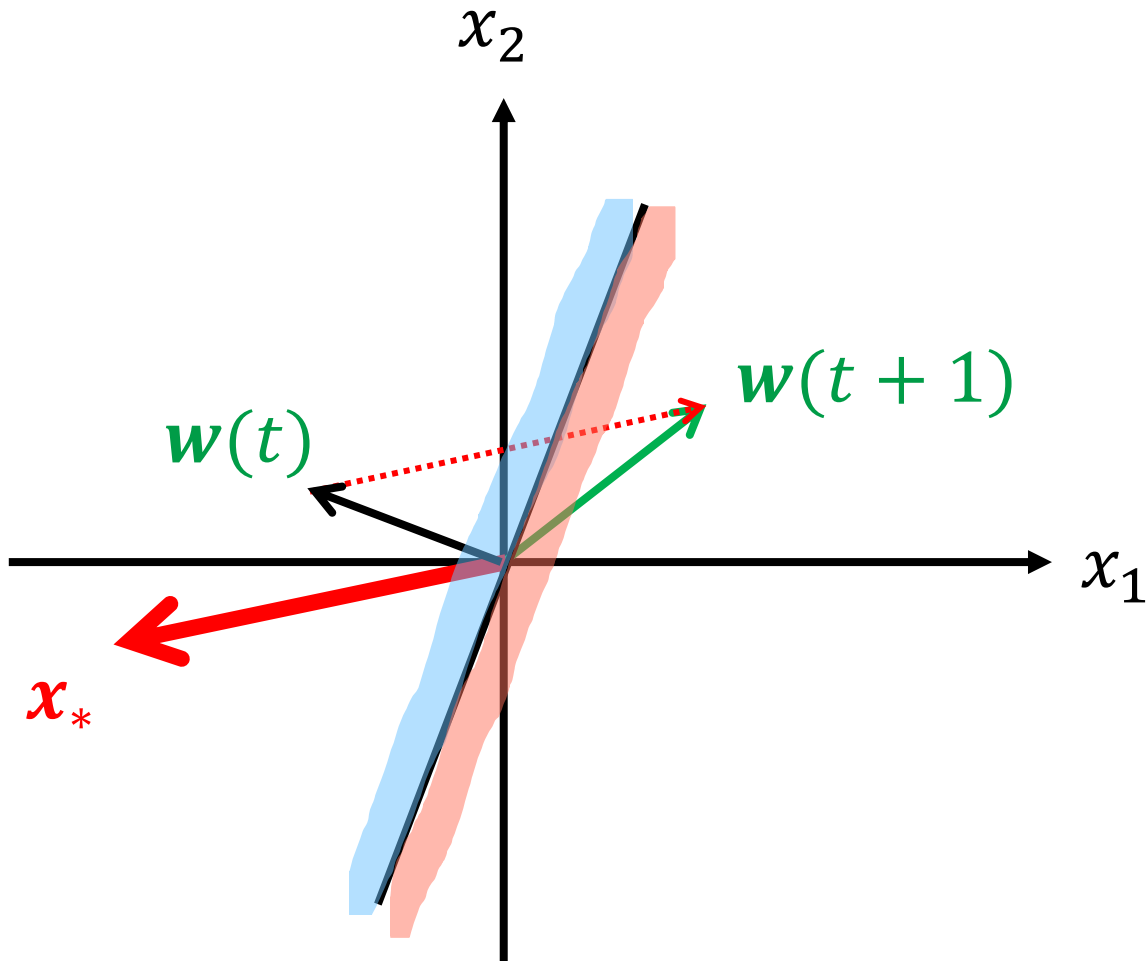
The Perceptron Update Rule

$$y_* = +1$$



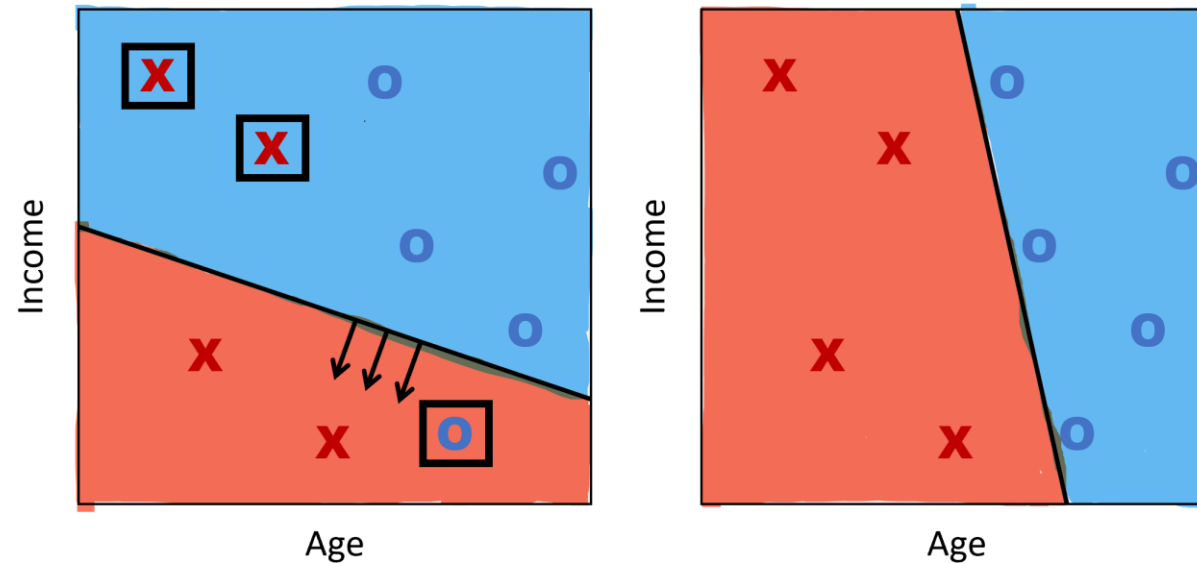
The Perceptron Update Rule

$$y_* = -1$$



Does PLA Work?

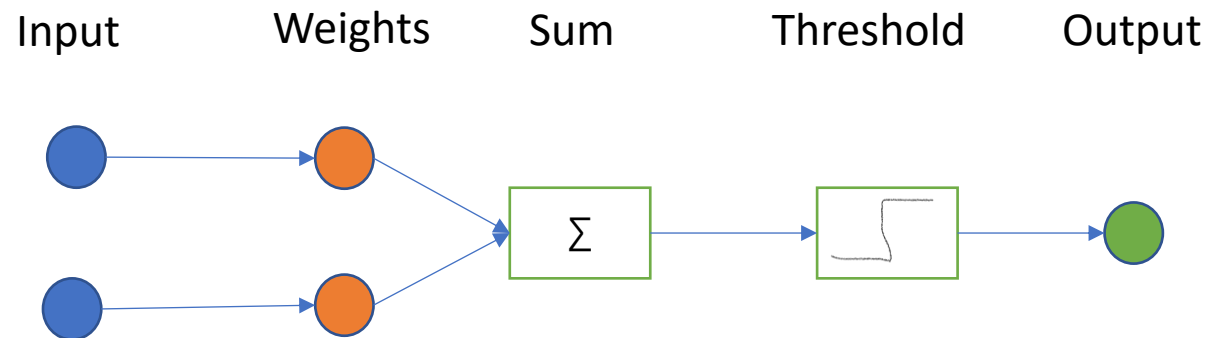
Theorem: If the data can be fit by a linear separator,
then after some *finite* number of steps, PLA will find one



- Want: $g \approx f$
 - But $g \approx f$ only on $\mathcal{D}$ (memorization)
 - While we want $g \approx f$ outside $\mathcal{D}$

Working Example of Learning:

location (X_1)	Time (X_2)	Fraud
0	0	-1
0	1	1
1	0	1
1	1	1



Working Example of Learning:

- 1st example: $X_1 = 0$ and $X_2 = 0$

$$\text{Sum} = W_1 * X_1 + W_2 * X_2 - 0.5$$

Let's Assume $W_0 = -0.5$, $W_1 = 0.4$ and $W_2 = 0.4$

$$\text{Sum} = 0 * 0.4 + 0 * 0.4 - 0.5 = -0.5$$

Output = -1 (Since the threshold function is 0)

Working Example of Learning:

- 2nd example: $X_1 = 0$ and $X_2 = 1$

$$\text{Sum} = W_1 * X_1 + W_2 * X_2 - 0.5$$

Now, $W_1 = 0.4$ and $W_2 = 0.4$

$$\text{Sum} = 0 * 0.4 + 1 * 0.4 - 0.5 = -0.1$$

Output = -1 (Since the threshold function is 0) (**Wrong**)

Working Example of Learning:

$$\begin{aligned}W_1 (\text{new}) &= W_1 (\text{old}) + 0.2 * 0 * 1 \text{ [learning rate} = 0.2\text{]} \\&= 0.4 + 0.2 * 0 * 1 \\&= 0.4\end{aligned}$$

$$\begin{aligned}W_2 (\text{new}) &= W_2 (\text{old}) + 0.2 * 1 * 1 \\&= 0.4 + 0.2 * 1 * 1 \\&= 0.6\end{aligned}$$

Working Example of Learning:

- 3rd example: $X_1 = 1$ and $X_2 = 0$

$$\text{Sum} = W_1 * X_1 + W_2 * X_2 - 0.5$$

Now, $W_1 = 0.4$ and $W_2 = 0.6$

$$\text{Sum} = 1 * 0.4 + 0 * 0.6 - 0.5 = -0.1$$

Output = -1 (Since the threshold function is 0) (**Wrong**)

Working Example of Learning:

$$\begin{aligned}W_1 (\text{new}) &= W_1 (\text{old}) + 0.2 * 1 * 1 \text{ [learning rate} = 0.2\text{]} \\&= 0.4 + 0.2 * 1 * 1 \\&= 0.6\end{aligned}$$

$$\begin{aligned}W_2 (\text{new}) &= W_2 (\text{old}) + 0.2 * 0 * 1 \\&= 0.6 + 0.2 * 0 * 1 \\&= 0.6\end{aligned}$$

Working Example of Learning:

- 4th example: $X_1 = 1$ and $X_2 = 1$

$$\text{Sum} = W_1 * X_1 + W_2 * X_2 - 0.5$$

Now, $W_1 = 0.6$ and $W_2 = 0.6$

$$\text{Sum} = 1 * 0.6 + 1 * 0.6 - 0.5 = 0.7$$

Output = 1 (Since the threshold function is 0)

Working Example of Learning:

- 2nd example: $X_1 = 0$ and $X_2 = 1$

$$\text{Sum} = W_1 * X_1 + W_2 * X_2 - 0.5$$

$$\text{Now, } W_1 = 0.6 \text{ and } W_2 = 0.6$$

$$\text{Sum} = 0 * 0.6 + 1 * 0.6 - 0.5 = 0.1$$

Output = 1 (Since the threshold function is 0) (**Correct**)

Working Example of Learning:

- 3rd example: $X_1 = 1$ and $X_2 = 0$

$$\text{Sum} = W_1 * X_1 + W_2 * X_2 - 0.5$$

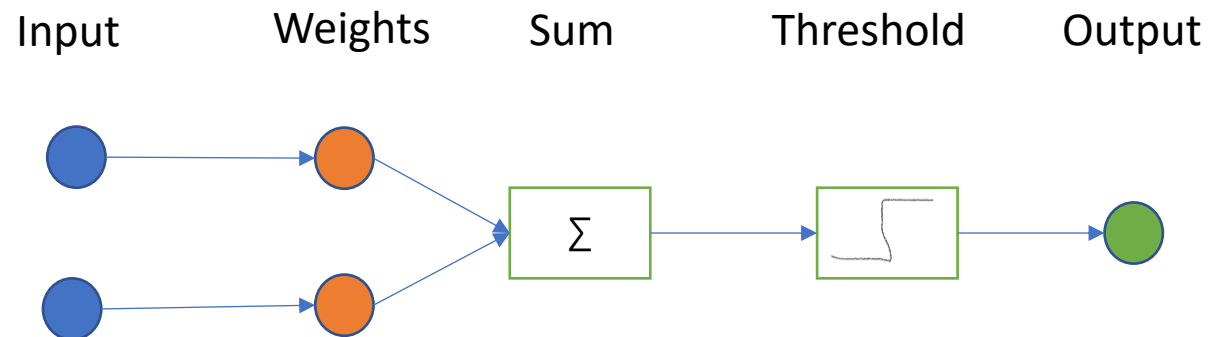
$$\text{Now, } W_1 = 0.6 \text{ and } W_2 = 0.6$$

$$\text{Sum} = 1 * 0.6 + 0 * 0.6 - 0.5 = 0.1$$

Output = 1 (Since the threshold function is 0.5) (**Correct**)

Working Example of Learning-2:

Input-1 (X_1)	Input-2 (X_2)	Output
1	2	1
2	1	-1
0	-1	-1



Working Example of Learning-2:

- 1st example: $X_1 = 1$ and $X_2 = 2$

$$\text{Sum} = W_1 * X_1 + W_2 * X_2 - 0.5$$

Let's Assume $W_0 = -0.5$, $W_1 = -0.2$ and $W_2 = 1$

$$\text{Sum} = -1 * 0.2 + 2 * 1 - 0.5 = 1.7$$

Output = 1 (Since the threshold function is 0)

Working Example of Learning-2:

- 2nd example: $X_1 = 2$ and $X_2 = 1$

$$\text{Sum} = W_1 * X_1 + W_2 * X_2 - 0.5$$

Let's Assume $W_0 = -0.5$, $W_1 = -0.2$ and $W_2 = 1$

$$\text{Sum} = -2 * 0.2 + 1 * 1 - 0.5 = 0.1$$

Output = 1 (Since the threshold function is 0) (**wrong**)

Working Example of Learning-2:

$$\begin{aligned}W_1 (\text{new}) &= W_1 (\text{old}) - 0.1 * 1 * 2 \text{ [learning rate} = 0.1\text{]} \\&= -0.2 - 0.2 \\&= -0.4\end{aligned}$$

$$\begin{aligned}W_2 (\text{new}) &= W_2 (\text{old}) - 0.1 * 1 * 1 \\&= 1 - 0.1 \\&= 0.9\end{aligned}$$

Working Example of Learning-2:

- 3rd example: $X_1 = 0$ and $X_2 = -1$

$$\text{Sum} = W_1 * X_1 + W_2 * X_2 - 0.5$$

Let's Assume $W_0 = -0.5$, $W_1 = -0.4$ and $W_2 = 0.9$

$$\text{Sum} = -0 * 0.4 - 1 * 0.9 - 0.5 = - 1.4$$

Output = -1 (Since the threshold function is 0)

Summary

- Applications of Machine Learning (ML) is almost everywhere in our daily life
- Data, Model (Hypothesis) and Learning Algorithms are critical for ML
- A practical example of ML is credit card approval rate
- Perceptron is a simple learning algorithm which can be learned through a simple update rule

References and Further Reading

- Zhang, Aston, Zachary C. Lipton, Mu Li, and Alexander J. Smola. "Dive into deep learning." *arXiv preprint arXiv:2106.11342* (2021).
- Bi, Qifang, Katherine E. Goodman, Joshua Kaminsky, and Justin Lessler. "What is machine learning? A primer for the epidemiologist." *American journal of epidemiology* 188, no. 12 (2019): 2222-2239.
- Jordan, Michael I., and Tom M. Mitchell. "Machine learning: Trends, perspectives, and prospects." *Science* 349, no. 6245 (2015): 255-260.
- Wang, H., Z. Lei, X. Zhang, B. Zhou, and J. Peng. "Machine learning basics." *Deep learning* (2016): 98-164.